# MedEx: Senior Frontend Engineer Take-Home Exercise

## Task 1: Service Explorer with Live Faceted Filters

Thank you for your interest in joining MedEx.

MedEx is a healthcare marketplace. Customers in Thailand, Nepal and India use it to find clinics, hospitals and labs, compare health packages, lab tests and wellness treatments, and book them online. On a single visit, one customer often deals with several providers and several currencies.

This exercise is about UI and UX when the data is messy. The data we give you has the same shape as our production API, rough edges included. How you handle those edges, and how you explain your decisions, matters much more to us than how many features you finish.

## How this works

- **Time:** spend about 6–8 hours. You have 5 days from the day you receive this. If you run out of time, stop and write in your README what you would do next. That counts in your favour.
- **Stack:** React with TypeScript is required. Next.js is preferred, since it's what we use, but not required. Use any component library, styling approach or state tool you like, and tell us why you chose it.
- **Data:** the task's JSON file is in the `data/` folder next to this brief. All of it is fictional. Don't import the JSON straight into components. Put it behind your own mock API layer, for example MSW, Next.js API routes, or an in-memory service. That layer should have realistic latency, so your UI works the way it would against a real server. The field names and formats match our real API, so expect to map them into the shapes your UI needs.
- **AI tools** are allowed. If you use them, say where in your README. You must be able to explain every line in the follow-up interview.

## What to send us

1. A link to a Git repository (GitHub or GitLab) with a commit history we can follow.
2. A README.md covering:
3. how to run it (ideally with one command),
4. the decisions you made and the trade-offs you accepted,
5. anything you deliberately left out, and what you would do with more time,
6. where you used AI tools, if you did.
7. Tests for the logic you consider most important. We don't expect full coverage.
8. Optional but welcome: a deployed link, a 3–5 minute screen recording walking through it, or both.

## How we evaluate

| Area | What we look for |
|---|---|
| UX thinking | Every state makes sense: loading, empty, error, partial, slow and offline. A non-technical customer understands what is happening. |
| Correctness | The logic is right in the awkward cases in the data, not just the happy path. |
| Architecture | State lives where it belongs, and a teammate could extend your code without fear. |
| Accessibility | Keyboard use, screen readers, focus management and contrast. |
| Performance | It stays smooth with realistic data volume. |
| Communication | Your README is honest and clear about trade-offs. |

In the follow-up interview, you'll walk us through your solution and then extend it live for about 30 minutes.

## Domain notes

**Service types.** Every service has a `type`: `package`, `lab_test`, `wellness`, `service` or `insurance`.

**Money is sent as strings.** Like our real API, the data stores `price`, `discount` and `serviceCharge` as decimal strings, for example `"6200.00"`. Parse them carefully.

**Currency belongs to the provider.** A service is priced in its provider's currency (`providerInformation.currency`: THB, NPR or INR). Our system has no exchange rates. Never add amounts in different currencies together, and never convert silently.

**Final price.** Unless the service is wellness (see the next note), the customer pays:

```
discountAmount = discountType === 'percent' ? price × discount / 100 : discount
afterDiscount  = price − discountAmount
chargeAmount   = serviceChargeType === 'percent' ? afterDiscount × serviceCharge / 100 : serviceCharge
finalPrice     = afterDiscount + chargeAmount
```

Round only for display, to 2 decimal places, never in the middle of a calculation.

**Wellness services have no single price.** Their price depends on a duration option the customer picks, such as 60 or 90 minutes. For these services `price` is `null`, and the options are listed in `wellnessPricing`. Each option uses the same formula as above. A wellness service's price is unknown until an option is chosen.

**Categories are a two-level tree** (category, then subcategory). Both levels have a `type`. A category `slug` is unique only within its type, so two top-level categories can share a slug when their types differ.

**What customers may see.** A service is visible only if all three of these hold: its own `status` is `true`, its provider's `status` is `"active"`, and its category (if it has one) has `status: true`.

Providers enter this data by hand. It contains mistakes and inconsistencies. When you find one, decide what the customer should see, handle it, and write the decision down in your README.

## Your task

**Data file:** `data/task1-service-explorer.json`

### The story

> *"I need a full-body check-up near Bangkok, under 5,000 THB, and I'd like to compare a few hospitals side by side before I commit."*

Build the page a customer uses to browse and filter services, grouped by provider.

### Data

The file contains `categories` (13 top-level categories with their children), `providers` (67) and `services` (3,200). Serve each as its own mock endpoint. Here is an excerpt showing every field:

```
// categories[] (shape of GET /category/active)
{ "id": "…", "title": "Ultrasound", "slug": "ultrasound", "type": "package", "position": 2,
  "status": true, "parentId": null,
  "children": [ { "id": "…", "title": "Thyroid", "slug": "thyroid", "type": "package",
                  "position": 1, "status": true, "parentId": "…" } ] }

// providers[]
{ "id": 48, "name": "Riverside International Hospital", "username": "riverside-international-hospital",
  "status": "active",
  "providerInformation": {
    "companyName": "Riverside International Hospital", "slug": "riverside-international-hospital",
    "types": ["package", "lab_test", "service"], "currency": "THB",
    "city": { "id": "…", "name": "Bangkok", "slug": "bangkok" },
    "country": { "id": 219, "name": "Thailand", "alphaCode": "TH" },
    "rating": 4.7, "reviewCount": 1203 } }

// services[] (shape of GET /service; userId is the provider id)
{ "id": "5f0c1a52-6b1e-4c1f-9a51-000000000001", "title": "Executive Health Check (Male)",
  "slug": "executive-health-check-male-5f0c", "type": "package", "userId": 48,
  "categoryId": "…", "subCategoryId": "…",
  "price": "6200.00", "discount": "10.00", "discountType": "percent",
  "serviceCharge": "7.00", "serviceChargeType": "percent",
  "status": true, "testAtHome": false, "homeTestCharge": null,
  "wellnessPricing": [], "createdAt": "2025-03-25T04:00:00.000Z" }

// a wellness service: no price until a duration is chosen
{ "id": "5f0c1a52-6b1e-4c1f-9a51-000000000003", "title": "Signature Facial", "type": "wellness",
  "userId": 61, "price": null, "discount": null, "discountType": null,
  "serviceCharge": null, "serviceChargeType": null,
  "wellnessPricing": [
    { "id": "…", "durationInMinutes": 60, "price": "1800.00", "discount": "0.00", "discountType":
"amount",
      "serviceCharge": "0.00", "serviceChargeType": "amount" },
    { "id": "…", "durationInMinutes": 90, "price": "2400.00", "discount": "15.00", "discountType":
"percent",
      "serviceCharge": "0.00", "serviceChargeType": "amount" } ], … }
```

## Requirements

1. **Results grouped by provider.** Each provider card shows the provider and its rating, with a sensible state when there is no rating yet. It shows at most 3 of that provider's matching services, plus a clear way to see the rest (for example "View all 14 services"). That count must reflect the active filters.

2. **Filters:**

3. Service type (multi-select)
4. Category, then subcategory (a tree: choosing a category makes its subcategories available)
5. Provider (searchable multi-select)
6. City or country
7. Price range
8. "Available as a home test"

9. Free-text search across service title, provider name and category

10. **Live facet counts.** Each filter option shows how many results it would lead to, given all the other active filters. For example, with "Bangkok" selected, the "Lab test" option shows how many lab tests are available in Bangkok. Options that would lead to zero results must be clearly unavailable. Decide whether to hide or disable them, and justify the choice.

11. **Prices.**

12. Show the final price, calculated with the formula in the domain notes. When there is a discount, show the original price struck through.
13. Wellness services have no single price. Show something honest and useful, for example the lowest option, clearly labelled.

14. The price-range filter must not mix currencies. Design how a customer filters by price when the results span THB, NPR and INR, and explain your design.

15. **Sorting:**

16. relevance (you define it, and explain your definition)
17. price, low to high
18. price, high to low

19. rating

20. **The URL is the source of truth.** Every filter, the sort order and the page number live in the URL. Reloading, sharing a link and using the browser's back and forward buttons must all restore exactly the same view. If you use Next.js, the first render must not flash an unfiltered list.

21. **Mobile.** On small screens, filters open in a drawer. While the drawer is open, its "Show results" button must say how many results the pending selection will produce, before the customer applies it.

22. **Performance.** With all 3,200 services loaded:
23. changing a filter updates the results and facet counts within about 100 ms on a mid-range laptop;
24. typing in any search field never blocks input, even for a single keystroke;
25. results updating causes no layout shift.

Tell us in your README how you measured this.

1. **States:**
2. Loading: use skeletons, not spinners.
3. Empty results: offer a way out, for example "Remove the price filter".
4. API failure: allow a retry. Make your mock API able to fail on demand so we can see this state.
5. **Accessibility.** Everything works with the keyboard alone, and screen readers announce when the number of results changes.

### Hints on what we will look at closely

- How you compute facet counts, and whether they stay correct when several filters are active.
- The two top-level categories that share the slug `ultrasound`.
- Your design for prices across several currencies.
- What you do with the data's mistakes.

## Questions?

If something is genuinely ambiguous, make a reasonable decision, write it down in your README and carry on. That is exactly what we'd want on the job. If you're blocked, email us at **[ADD HIRING EMAIL]** and we'll reply within one working day.

Good luck. We're looking forward to seeing how you think.